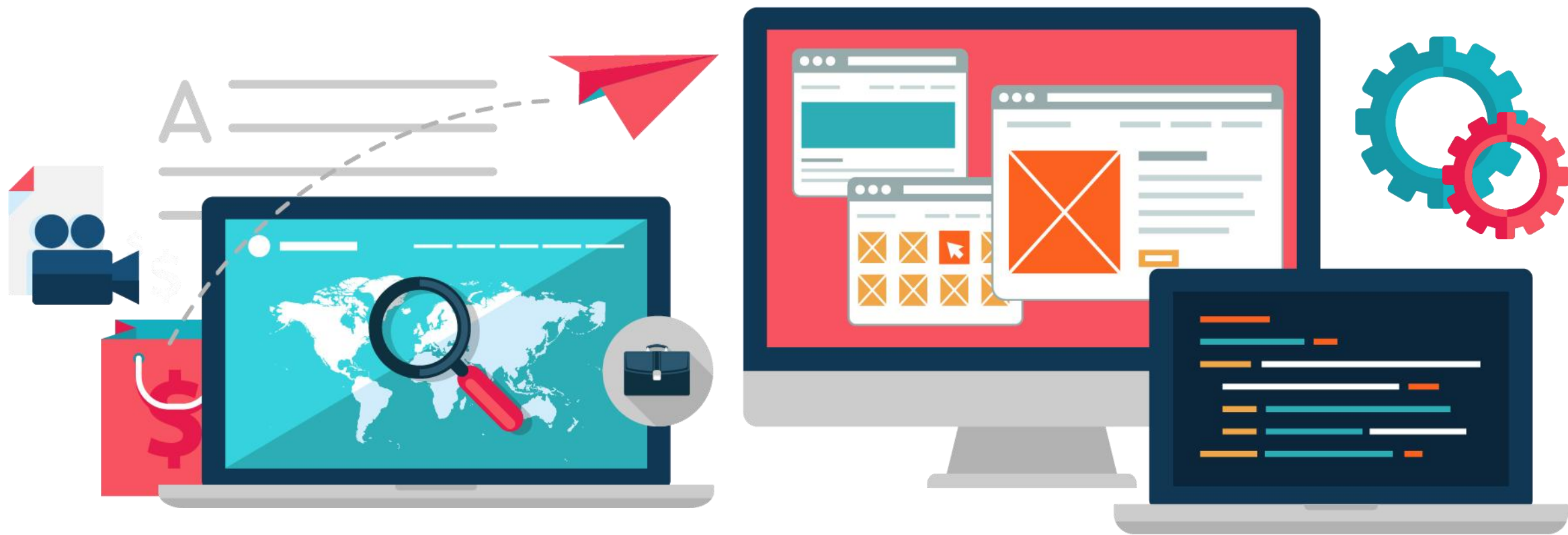


面向对象程序设计

Object Oriented Programming





继承和类型转换



01

protected/private继承下的类型转换

02

public继承下的向上类型转换

03

public继承下的向下类型转换

04

类型转换操作符





//基类和派生类如下

```
class Base
```

```
{
```

```
    //....
```

```
};
```

```
class Derived
```

```
    : public Base
```

```
{
```

```
    //.....
```

```
};
```

- ◆ 向上（下）类型转换：将派生类（基类）的指针、引用或对象转换成基类（派生类）的指针、引用或对象。
- ◆ protected/private继承下
 - ◆ 从派生类向基类转换：向上类型转换
 - ◆ 从基类向派生类转换：向下类型转换
- ◆ public继承下
 - ◆ 从子类向父类转换：向上类型转换
 - ◆ 从父类向子类转换：向下类型转换



//基类和派生类如下

```
class Base
```

```
{
```

```
    //....
```

```
};
```



```
class Derived
```

```
    : private Base
```

```
{
```

```
    //.....
```

```
};
```

保护和私有继承下(protected、private):

从派生类向基类转换:

- 基类的公有成员在派生类中变为protected或private, 因此从外部(非派生类成员、非友元)无法进行从派生类到基类的向上转换(static_cast 不可用), 对类的使用者来说无实际意义。但在派生类成员函数或友元内部, 这种转换仍然允许。
- 如果确实需要对外部提供转换, 可以在派生类中定义转换函数(如 operator Base&() const)或在基类中定义转换构造函数(如 Base::Base(const Derived& d))来安全实现, 尽量避免使用强制类型转换。



```
class Bike {  
public:  
    void move( );  
    void stop( );  
    void repair( );  
protected:  
    int  changeColor(int );  
private:  
    int  mColor;  
};
```

```
int main( ) {  
    Player player;  
    Bike * pb = (Bike * ) (&player);  
    pb->move( );  
    constBike & b = (Bike ) (player); //error  
}
```

```
class Player : private Bike {  
public:  
    void  startRace( )      { /*....*/ }  
    void  endRace( )        { /*....*/ }  
protected:  
    int  curStrength ( );  
private:  
    int  mMaxStrength;  
    int  mAge;  
};
```

1.此时向上类型转换对类使用者无实际意义
2.可通过用户定义转换(转换构造函数或转换函数)进行转换



//基类和派生类如下

```
class Base
```

```
{
```

```
    //....
```

```
};
```



```
class Derived
```

```
    : protected
```

```
    Base
```

```
{
```

```
    //.....
```

```
};
```

保护和私有继承下(protected、private):

从基类向派生类转换:

- Base类和Derived类中的public互不相同, 向下类型转换无实际意义(行为集不同, 数据也不全)
- 若确实需要, 可在Derived中定义转换构造函数
Derived::Deived(const Base&);或在Base类中定义静态工厂方法(技术可行但不推荐)



```
class Car {  
public:  
    void run( );  
    void brake( );  
protected:  
    int  addWeight(int w );  
    int  reduceWeight(int w );  
private:  
    int  mWeight;  
    int  mWheels;  
};
```

```
class MTCar:public Car {  
public:  
    void funcMT( );  
protected:  
    //...  
private:  
    //....  
};
```

public继承方式下

1. 此时的向上类型转换有意义
2. 逻辑上，是类型的泛化或一般化
3. 语言上，public行为集被窄化



//子类的指针转换成父类指针

MTCar c;

Car * c1 = &c; //安全的

Car * c2 = new MTCar; //安全的

◆指针型：安全的

◆引用型：安全的

◆对象型：安全的，但进行了对象裁剪。

//子类对象转换成父类引用

MTCar c;

Car & c1 = c; //安全的

const Car & c2 = c; //安全的

◆结论:应尽可能使用指针或引用型向上类型转换。

//子类转换成父类对象

MTCar c;

(Car) c; //显式类型转换，创建临时对象(纯右值)，没有名字，表达式结束后立即析构

//将c转换成父类对象是安全的，但转换后的对象是

//一个新对象，它与c不是同一个对象。这个新对象是经过裁剪的



```
class Example
{
public:
    Example( ) { mp = new Child; }
    ~Example( ) {
        //要保证正确释放mp,
        // Parent类的析构函数应为虚函数
        delete mp;
    }
    int func( Parent & obj)
        { obj.doFunc( ); }
private:
    Parent * mp;
};
```

```
class Parent {
public:
    virtual ~Parent() {}
    virtual void doFunc( );
    //....
};
class Child: public Parent{
    //.....
};
```

```
int main( ) {
    Example exam;
    Child child;
    Parent & parent = child;
    exam.func(parent);
    exam.func(child);
}
```



```
MTCar aMTCar;
ATCar aATCar;
Car * pCar= &aMTCar;

void func(Car * pCar) {
    MTCar * p1=(MTCar*)pCar;
    ...
}
```

public继承方式下向下类型转换

1. 此时的向下类型转换有意义
2. 但可能成功，也可能失败
3. 只能在运行时确定(dynamic_cast)



转换方式

1. 内置类型的自动转换, 如 `int->float`
2. 构造函数转换
3. 定义自动转换函数
4. `public`继承下的向上类型自动转换
5. 使用类型转换操作符
 1. `static_cast`转换操作符
 2. `const_cast`转换操作符
 3. `reinterpret_cast`转换操作符
 4. `dynamic_cast`转换操作符



格式: `static_cast<T>(exp)`

静态转换: 用隐式和用户定义转换的组合在类型间转换。

如: `void fun(float a); void fun(double b);`
`int n=10; fun(static_cast<float>(n));`
`float x = 2.5f;`
`cout << static_cast<int>(x) << endl; //ok`
`cout << static_cast<string>(x) << endl; //error`
`cout << static_cast<string>("helloworld") << endl; //ok`
`A * pA = new A;`
`char * pByte = static_cast<char *>(pA); //error`
`void * pv = pA;`
`char * pByte = static_cast<char *>(pv); //ok`

`static_cast<T>(exp)`等价于C语言中的强制类型转换 `(T)exp?`



格式: `const_cast<T>(exp)`

常量转换: 用于添加或移除表达式中的`const`或`volatile`约束

说明: 其中 T 与exp的类型, 除了`const`或`volatile`等修饰之外, 必须一致

```
class A {
public:
    A(int n):val(n) { }
    void setVal(int n)
        { val = n;}
    int  getVal( ) const
        { return val; }
    int  getVal( )
        { return val+1; }
private:
    int val;
};
```

```
int main( ) {
    A      va(3);
    const A & ca = va;
    cout<<ca.getVal(); // 输出 3
    cout<<va.getVal(); // 输出 4
    va.setVal(5);      //ok,val改为5
    ca.setVal(5);      //compile error

    A & var = const_cast<A & > ( ca );
    var.setVal(6);      //ok,val改为6
}
```



volatile关键字是一种类型修饰符，用它声明的变量表示可以被某些编译器未知的因素更改，比如：操作系统、硬件或者其它线程等。遇到这个关键字声明的变量，编译器对访问该变量的代码就不再进行优化，系统总是重新从它所在的内存读取数据。

```
class A {
public:
    A(int n):val(n) { }
    void  Func1( );
    int   Func2(volatile int & k);
private:
    volatile int  val;
    int          other;
    ...
};
```

```
void A::Func1( ) {
    val = 1;
    val = 2;
    val = 3;
}
int A::Func2(volatile int & k) {
    ///实现1
    return k*k;
    ///实现2
    int temp = k;
    return temp*temp;
}
```



```
A *      p1 = new A;  
const A * const p2 = p1;  
const A *      p3 = const_cast< const A * >(p2);  
A * const p4 = const_cast< A * const> (p2);  
A *      p5 = const_cast<A *>(p2);
```

```
//例  
class A {  
public:  
    A(int n):val(n) { }  
    int  getVal( ) const;  
private:  
    int  val;  
    int  other;  
    ...  
};
```

```
int  A::getVal( ) const  
{  
    A * const p = const_cast<A * const>(this);  
    p->val = xxx;  
    return val;  
};
```



格式: `reinterpret_cast<T>(exp)`

重新解释: 对表达式的类型做出重新解释, 常用于重新解释函数

例:

```
int  MyFunc( void * pt ) { /*...*/ }
```

```
typedef void (* FuncType) (char *);  
FuncType f = reinterpret_cast<FuncType >(MyFunc);
```

```
char * buf = new char[256];  
f(buf);  
delete[ ] buf;
```


格式: `dynamic_cast<T>(exp)`

动态转换:

- 在运行时刻, 尝试将exp转换成T类型。多用于public继承下, 将父类类型转换成子类类型;
- T **只能**为类的指针、类的引用、`void *`三种形式;
- `dynamic_cast`要用到RTTI(运行时类型识别), 通常各编译器都是通过虚拟表(VTable)来实现RTTI的, 因此类中要有虚函数, 才能使用。

//T为指针型

```
class A { public: virtual ~A(){ } };  
class B: public A { };  
class C: public A { };
```

```
A * p = new B;    //或 A * p = new C;  
B * pb = dynamic_cast<B*>(p);  
if( pb ) {  
    pb->....  
} else {  
}
```

//T为引用型

```
class A { public: virtual ~A(){ } };  
class B: public A { };  
class C: public A { };
```

```
A * p = new B;    //或 A * p = new C;  
try {  
    //下句若失败, 产生运行时异常  
    B & b = dynamic_cast<B&>(*p);  
} catch(...) {  
}
```



本章结束